Computational equality: six rules for iPRA

A smaller presentation on the original scalar syntax

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current raw witness presentation of ConciseIPRA has seven inference constructors. Three of them provide zero computation, successor computation, and equality transport in an arbitrary numerical template. We replace those three by reflexivity and one deterministic rule that combines computation with equality of a preceding iterator value. The resulting six-rule presentation uses exactly the original raw terms and native formulas: there is no canonical term grammar, normalizing substitution, new logical meaning, or extra induction principle. Both original computation equations have three-node derivations. Arbitrary numerical congruence is recovered by an explicit one-iteration construction; the given equality proof occurs once. We prove exact equivalence for every retained context and formula, so both full iPRA interpretations, both formula composites, whole numerical domains, actual equality, and equiconsistency are inherited unchanged. A fully validated standalone Python checker decreases from 3121 to 3039 normalized bytes in a matched comparison, from 1008 to 959 significant tokens, and from 1064 to 1026 AST nodes. Maximum-compression gzip sizes are reported separately. This is a proved local simplification, not a theorem of absolute minimality or a new record for the repository's different compact certificate encodings.

1 The result and its comparison boundary

The comparison repository is `VladimirReshetnikov/ProveIt`, at the frozen commit

`11be5aabc6b07262046e2553419bc6b05271d178`.

The relevant directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README describes the native scalar calculus, the seven-rule witness presentation, the Lean interpretation chain, forty-three research packages, and separate families of Python encodings [1]. We also inspected the six incoming reports present at that commit. In particular, zero-only normalization, template-free rewriting, and initializer-only equality had already been proposed [4].

The contribution here is a different arithmetic/equality interface. *The major equality of a successor computation names the preceding iterator value.* This permits the update body to act directly on the new value. Both arithmetic computations and equality transport follow from the same rule, without a supplied template.

Feature	Raw witness calculus $\mathcal{W}$	New calculus $\mathcal{E}$
Terms and formulas	Raw scalar terms; $=, \mathsf{Q}, \mathsf{D}$	Identical
Arithmetic/equality rules	$I0$, IS, one-template Tr	Refl, Eval
Other rules	Hyp, QE, DE, DI	Identical
Total inference constructors	Seven	Six
Template supplied to equality rule	One term template	None
Computation-only axiom constructors	Two	None
Normalization during substitution	None	None

The computational content is explicit in Eval's three cases. Removing separate axiom constructors does not erase that content, and we count its implementation in every checker measurement. Six-rule presentations with different term grammars already exist. The stronger point here is obtaining six rules *while preserving the original raw syntax.*

2 Complete syntax and primitive rules

2.1 Raw numerical syntax

There is one numerical sort. Terms are finite trees generated by

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound by the iterator constructor. The counter n and initializer a are outside that binding. The update h may contain arbitrary passive numerical parameters and nested iterators. Every substitution in this paper is ordinary capture-free substitution; bound variables may be renamed first. No computation is performed by substitution. Thus $I(0, a; u.h)$ is a legal raw term distinct from a.

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The paired binder has distinct bound variables x, y. Contexts Γ are finite lists of reusable assumptions. The designated contradiction is

$$\bot_* \ := \ (0 = S0).$$

An equation is *literal* when its outermost formula constructor is equality. An equation beneath Q or D is not a literal equation.

The unchanged ordinary readings of the two binders are

$$(\mathsf{Q}_y B(y))^\circ := B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B(x,y))^\circ := \exists x \big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big).$$

These readings explain the calculus; they are not additional inference rules or a hidden appeal to a stronger background object theory.

2.2 The computational equality operation

Write $b \equiv s$ for literal identity of finite syntax, up to the chosen representation of bound names. In the implementation, de Bruijn indices make this plain tree equality.

Definition 2.1. For numerical terms s, t, b, define the partial operation $\Phi(s, t, b)$ by the following disjoint cases:

shape of b	required source s	$\Phi(s,t,b)$
s	s	t
$I(0, a; u.h)$	a	t
$I(Sn, a; u.h)$	$I(n, a; u.h)$	$h[t/u]$.

(Φ)

The first row means $b \equiv s$. Each source comparison in the other rows is also literal. The operation is undefined otherwise.

In the zero row, a is a proper subterm of b, so that row cannot overlap the first. In the successor row, b has exactly one more successor node than its required source, so it too cannot overlap the first. The other two rows have distinct counter constructors. This is a deterministic partial operation, not a proof search procedure.

The successor row uses an equality for the *whole preceding iterator value* $I(n, a; u.h)$. It does not compare two initializers. In particular, it is valid even when n is literally 0, and it produces the raw substituted term $h[t/u]$. There is no special one-iteration shortcut.

2.3 The entire rule inventory

Define

$$\eta_{B,t}(z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

The six primitive rules of $\mathcal{E}$ are exactly the following:

$$\frac{A \in \Gamma}{\Gamma \vdash_{\mathcal{E}} A} \qquad \text{Hyp},$$

$$\frac{}{\Gamma \vdash_{\mathcal{E}} s = s} \qquad \mathsf{Refl},$$

$$\frac{\Gamma \vdash_{\mathcal{E}} s = t \qquad \Gamma \vdash_{\mathcal{E}} b = r}{\Gamma \vdash_{\mathcal{E}} r = \Phi(s,t,b)} \quad (\Phi(s,t,b) \text{ defined}) \qquad \mathsf{Eval},$$

$$\frac{\Gamma \vdash_{\mathcal{E}} \mathsf{Q}_y B(y) \qquad \Gamma \vdash_{\mathcal{E}} B(0)}{\Gamma \vdash_{\mathcal{E}} B(v)} \qquad \text{QE},$$

$$\frac{\Gamma \vdash_{\mathcal{E}} \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, \mathsf{Q}_y B(b,y) \vdash_{\mathcal{E}} A}{\Gamma \vdash_{\mathcal{E}} A} \qquad \text{DE},$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash_{\mathcal{E}} B(t, Sz)}{\Gamma \vdash_{\mathcal{E}} \mathsf{D}_{x,y} B(x,y)} \qquad \text{DI}.$$

In DE, b is a fresh numerical eigenvariable, absent from the retained context, conclusion, and passive parameters of B. In DI, z is fresh for the retained context, witness t, and passive parameters of B. Displayed local assumptions are discharged. All numerical instances are well-scoped terms. The letters v and t in QE and DI are arbitrary terms, not necessarily variables or numerals.

No other logical axiom, equality axiom, successor axiom, structural rule, or induction principle is primitive. The four unchanged rules Hyp, QE, DE, DI are precisely those of the repository's single-introduction witness calculus. The equality conclusion is deliberately crossed: its left side is the right side r of the second premise.

3 Recovering arithmetic and all equality transport

Every proof in this section works over an arbitrary retained context. It uses only Refl and Eval, apart from any explicitly supplied premise proofs. No logical binder rule, numerical case distinction, or induction is used.

Write R_s for the one-node proof $s = s$. We use Eval as a proof constructor $\mathsf{Eval}(p, q)$ whose first and second arguments supply its two premises.

Lemma 3.1 (Equality algebra)**.** *Symmetry and transitivity are derivable.*

Proof. The first case of Φ is the rule

$$\frac{s = t \qquad s = r}{r = t}.$$

For $p : s = t$, define $\mathrm{Sym}(p) = \mathsf{Eval}(R_s, p)$; its conclusion is $t = s$. For $p : s = t$ and $q : t = u$, the proof $\mathsf{Eval}(q, \mathrm{Sym}(p))$ concludes $s = u$. □

Lemma 3.2 (Both computation equations)**.** *For every raw n, a, h, the following equations have three-node proofs:*

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

Proof. Put $A = I(0, a; u.h)$, $C = I(n, a; u.h)$, and $B = I(Sn, a; u.h)$. The proofs are

$$\mathsf{Eval}(R_a, R_A) : A = a, \qquad \mathsf{Eval}(R_C, R_B) : B = h[C/u].$$

Each consists of two reflexivity leaves and one Eval node. The second construction is unchanged when $n \equiv 0$. □

The essential next lemma is more than a soundness check. It shows that the small fixed computation interface already contains arbitrary numerical congruence.

Theorem 3.3 (Arbitrary numerical-template congruence)**.** *For every fixed numerical template $h(u)$,*

$$\frac{\Gamma \vdash_{\mathcal{E}} s = t}{\Gamma \vdash_{\mathcal{E}} h[s/u] = h[t/u]}$$

is admissible. The template can have repeated occurrences of u, occurrences under nested iterator binders, and arbitrary passive parameters.

Proof. Given $p : s = t$, set

$$A_s = I(0, s; u.h), \qquad B_s = I(S0, s; u.h).$$

The following four constructions give the complete derivation:

$$\begin{aligned} z_s &:= \mathsf{Eval}(R_s, R_{A_s}) && : \quad A_s = s, \\ c_s &:= \mathsf{Eval}(z_s, R_{B_s}) && : \quad B_s = h[s/u], \\ z_{s,t} &:= \mathsf{Eval}(p, R_{A_s}) && : \quad A_s = t, \\ \mathrm{Cong}_h(p) &:= \mathsf{Eval}(z_{s,t}, c_s) && : \quad h[s/u] = h[t/u]. \end{aligned}$$

The first and third lines use the zero row of Φ. In the second and fourth lines, the predecessor source required by the successor row is literally A_s. Thus all source comparisons are exact, and no congruence has been assumed inside its own derivation. The substitutions into h are ordinary capture-free substitutions supplied by Eval itself. □

Corollary 3.4 (The old crossed transport rule)**.** *The rule*

$$\frac{\Gamma \vdash_{\mathcal{E}} s = t \qquad \Gamma \vdash_{\mathcal{E}} h[s/u] = r}{\Gamma \vdash_{\mathcal{E}} r = h[t/u]} \tag{Tr}$$

is admissible for all numerical templates and all source terms, including literal 0*.*

Proof. Apply the root case of Eval to $\mathrm{Cong}_h(p)$ and the second supplied premise. □

Proposition 3.5 (Explicit node cost)**.** *The congruence macro adds eight inference nodes to its input equality proof. The transport macro adds nine nodes to its two input proofs combined. Neither input proof is duplicated.*

Proof. The congruence tree has four reflexivity leaves and four Eval nodes in addition to its single occurrence of p. The two occurrences of R_{A_s} are separate trivial leaves. One more root Eval gives transport. □

This count says nothing about the size of the repeated term labels, serialized certificates, runtime, or shortest possible proofs. In particular, a larger proof certificate is an acceptable consequence of making the trusted rule interface smaller; the costs are measured separately.

4 Exact equivalence with the current raw calculus

Let $\mathcal{W}$ denote the existing raw witness calculus: Hyp, I0, IS, the one-template crossed rule Tr, QE, DE, and the same witness-sensitive DI. This is the presentation named `WPrf` in the pinned `NativeOneIntro.lean`; the theorem `wprf_iff_nprf` identifies it with the native kernel $\mathcal{N}$ [2].

Lemma 4.1 (New atomic rules are admissible in $\mathcal{W}$)**.** *Every Refl and Eval instance is derivable in $\mathcal{W}$ under the same context.*

Proof. For reflexivity of s, apply Tr with the identity template to two copies of $I(0, s; u.0) = s$. The result is $s = s$. Tr then derives symmetry, transitivity, and congruence by the elementary equality constructions.

For Eval, let its premises be $p : s = t$ and $q : b = r$. If $b \equiv s$, use Tr with the identity template. If $b = I(0, a; u.h)$ and $s \equiv a$, I0 gives $b = s$; combining it with p and q by equality reasoning gives $r = t$. If $b = I(Sn, a; u.h)$ and $s \equiv I(n, a; u.h)$, IS gives $b = h[s/u]$. Together with q this gives $h[s/u] = r$. Applying Tr with p and template h gives $r = h[t/u]$, exactly the required conclusion. □

Theorem 4.2 (Identity-language consequence equivalence)**.** *For every finite native context* Γ *and every raw native formula* A*,*

$$\Gamma \vdash_{\mathcal{E}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{W}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

The first two transformations are effective on finite proof trees and leave every end-sequent term, formula, and retained assumption unchanged.

Proof. For $\mathcal{E} \to \mathcal{W}$, replace Refl and Eval by the preceding admissible derivations and copy the other four rules. For $\mathcal{W} \to \mathcal{E}$, replace I0 and IS by the three-node derivations of (I0, IS), replace Tr by Corollary 3.4, and copy the other rules. These are structural inductions on finite proof trees. An unchanged DI node keeps exactly the same witness and syntactic mode; no derivation-substitution theorem or cut theorem is assumed in the construction. The final equivalence is the inherited `wprf_iff_nprf` theorem. □

Corollary 4.3 (A linear inference-node bound)**.** *If an old proof has* N *inference nodes, the displayed* $\mathcal{W} \to \mathcal{E}$ *translation has at most* $9N$ *nodes.*

Proof. Each computation leaf becomes three nodes. Each Tr node is replaced by the translated premises, each used once, plus nine nodes. Every other node is copied once. Induction on the old tree gives the bound. □

The bound is for this specified tree translation. It is not a bound on formula sizes or JSON bytes, and it is not an optimality assertion.

5 Substitution, contexts, and the remaining mode condition

Proposition 5.1 (The new equality rule is substitution-stable)**.** *If* $\Phi(s, t, b)$ *is defined, then for every admissible capture-free numerical substitution* σ*,*

$$\Phi(s\sigma, t\sigma, b\sigma) \equiv \Phi(s, t, b)\sigma.$$

The same clause of Φ *applies before and after substitution.*

Proof. The root case is immediate. In the zero case, substitution preserves the outer zero-headed iterator and its initializer remains the substituted source. In the successor case it gives

$$b\sigma = I(S(n\sigma), a\sigma; u.h\sigma),$$
$$s\sigma = I(n\sigma, a\sigma; u.h\sigma).$$

Here substitution into h acts on its passive parameters and retains u as bound. The standard capture-free substitution-composition law identifies $(h[t/u])\sigma$ with $(h\sigma)[t\sigma/u]$. No computation takes place during substitution. Even if $n\sigma$ is literally zero, the preceding iterator stays the raw term $I(0, a\sigma; u.h\sigma)$. The size differences noted after (Φ) exclude collapse into the root case. □

This proposition concerns the new equality subsystem. The complete calculus deliberately retains the old witness-sensitive DI. A substitution can turn a witness into literal 0 and change that rule's mode. Thus a claim that arbitrary whole proofs substitute by simply reusing every rule tag would

still be false. Full admissibility of numerical substitution follows through Theorem 4.2 and the native substitution theorem. The same argument transfers weakening, cut, full formula transport, numerical cases, and contextual equation induction. The new Eval rule introduces no additional substitution repair.

All comparisons are of derivability under arbitrary retained assumptions, not only of closed theorems. This is why replacing local proof fragments is sufficient for the full interpretation and structural-rule claims.

6 Full iPRA, both interpretations, and both composites

6.1 The target theory and inherited bridge

Here iPRA means full first-order intuitionistic primitive recursive arithmetic: intuitionistic equality logic, symbols for fixed primitive recursive definitions with their defining equations, successor separation and injectivity, and induction for quantifier-free formulas. Quantifiers and intuitionistic implications are retained in the full object theory. This is the repository's target `PR` theory, not merely its equational or universal fragment [1, 3].

The existing bridge factors through the ordinary scalar theory T_{eq}. That theory has intuitionistic equality logic over the same iterator terms, both computation equations, numerical coverage

$$\forall x(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and contextual induction for literal equations. The native logical macros recover its full logic and these arithmetic principles. In T_{eq}, the inherited arithmetic development constructs a predecessor, equality reflection, quantifier-free induction, pairing, and a compiler for fixed primitive recursive definitions. Conversely, iPRA interprets each fixed scalar iterator by primitive recursion. The existing Lean development proves both formula round trips, not merely relative consistency [1, 3].

These are inherited results. The new proof obligation in this report is Theorem 4.2. Its proof does not assume any new arithmetic bootstrap lemma. For auditability, the archive includes the relevant pinned Lean statements and the five inherited written bridge files. Those Lean files are reference extracts, not a separately rebuilt project.

6.2 Why the same interpretation is a bi-interpretation

Let J be the repository's map `NFm.toPRA`, and K its map `NFm.ofPRA`. Use exactly those maps for $\mathcal{E}$. For native A and native context Γ, the inherited theorem gives

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longleftrightarrow \quad J\Gamma \vdash_{\mathsf{iPRA}} JA.$$

For an iPRA formula φ and context Δ, it also gives

$$\Delta \vdash_{\mathsf{iPRA}} \varphi \quad \Longleftrightarrow \quad K\Delta \vdash_{\mathcal{N}} K\varphi.$$

Theorem 4.2 permits replacing every $\mathcal{N}$ here by $\mathcal{E}$.

Theorem 6.1 (Bi-interpretability and equiconsistency)**.** *The calculus $\mathcal{E}$, with its ordinary realization, is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domains are whole, interpreted equality is actual equality, and both numerical composite graphs are $x = y$.*

Proof. The two faithful derivation translations are the preceding maps J, K. The inherited native round trip says that KJA and A are interderivable under arbitrary contexts. Apply Theorem 4.2 to both directions to obtain the same assertion in $\mathcal{E}$. The inherited iPRA round trip already gives

$$\mathsf{iPRA} \vdash \varphi \leftrightarrow JK\varphi,$$

and is unchanged. The term maps, numerical domain formulas, and interpreted equality have not changed at all, so the inherited identity composite graphs remain $x = y$. Finally, the native contradiction is still $0 = S0$, and the inherited contradiction correspondence gives

$$\varnothing \vdash_{\mathcal{E}} 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

This proves equiconsistency as well. □

In particular, no restriction to standard numerals, quotient equality, retract of a larger domain, new choice principle, or strengthening to induction for arbitrary formulas is involved.

6.3 Where ordinary intuitionistic logic comes from

For orientation, the inherited logical construction uses the numerical selector

$$C(v, a, b) = I(v, a; u.b), \qquad C(0, a, b) = a, \quad C(Sv, a, b) = b.$$

The two displayed equations now follow from the derived computations. Structural recursion on fixed formula trees provides a selector $\mathrm{sel}(v, A, B)$ that is equivalent to A at 0 and to B at a successor, and a gate $\mathrm{gate}(v, A)$ that is provable at 0 and equivalent to A at a successor. Mixed formula depths are handled by the inherited binder-padding equivalences; this is external recursion on finite syntax, not an internal truth predicate.

One convenient macro inventory is

$$\begin{aligned} A \to_* B &:= \mathsf{Q}_y \,\mathrm{sel}(y, A, B), \\ A \wedge_* B &:= \mathsf{Q}_y \,\mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists_* x\, A &:= \mathsf{D}_{x,y} \,\mathrm{gate}(y, A), \\ A \vee_* B &:= \exists_* v \ \mathrm{sel}(v, A, B), \\ \forall_* x\, A &:= A(0) \wedge_* \mathsf{Q}_y A(y), \qquad \bot_* := 0 = S0. \end{aligned}$$

Here newly bound variables are fresh, and par is the parity iterator in the inherited construction. All ordinary natural deduction rules and both formula composites of these macros are proved there. They transfer to $\mathcal{E}$ because all native sequents transfer. These macros are explanatory derived notation, not additional primitives.

7 The complete checker and controlled measurements

7.1 Public input and trusted boundary

The complete checker is `code/checker.py`. It imports only Python's `json` and `sys`; it imports neither the control checker nor the proof translators. Its input is one JSON object with exactly the

keys `variables`, `context`, `goal`, and `proof`. It validates arbitrary open or closed sequents, the full raw term/formula grammar, each proof instruction, and every local scope. Success is exit status zero with exactly `ACCEPT` on standard output.

Terms are represented as

$$[\mathtt{v}, i], \quad [\mathtt{z}], \quad [\mathtt{s}, t], \quad [\mathtt{i}, n, a, h],$$

where the tokens stand for JSON strings. De Bruijn variable 0 is the nearest numerical binder. In a D matrix, index 0 is y, index 1 is x, and higher indices refer to the surrounding context. The iterator body adds one binder; its other arguments add none.

Proof instruction	Payload
`hyp`	An index in the retained assumption list
`refl`	One numerical term
`ev`	Two premise proofs
`qe`	A target term and two premise proofs
`de`	A result formula, a major proof, and a local branch
`di`	A two-variable matrix, a witness, and a local branch

The Eval instruction receives no occurrence path, template, update body, counter, or predecessor term separately. These are extracted from already proved equations. The checker does one state-variable substitution and one finite-tree comparison in the successor case. It does not normalize arbitrary primitive recursive programs or postulate a uniform total primitive recursive evaluator.

7.2 Why the omitted tag checks are justified

The key implementation invariant is: whenever `infer(p,g,n)` returns under a well-formed context, it returns a well-scoped native formula in n variables. This is proved by induction over successful recursive calls. Hyp selects a validated assumption; Refl validates its term; QE checks its matrix and base; DE checks its result and the shifted local conclusion; DI checks its two-variable matrix, witness, local hypothesis, and successor result. Eval constructs an equation from scoped terms and a capture-free substitution.

Under this invariant, only equality formulas have length three: Q and D formulas have length two. Thus triple destructuring of a proved formula checks that it is an equation. Similarly, only iterator terms have length four. These facts justify omitting repeated constructor-tag tests in Eval. The matched control uses exactly the same invariant to omit its corresponding redundant equality-tag tests.

This does not weaken public validation. Arbitrary JSON is checked before becoming an assumption or goal, and every syntax-bearing instruction is validated. Boolean values are explicitly excluded where integers are required. There are no Python `assert` gates: optimization mode does not disable rejection. As with any executable checker, physical memory and recursion limits can reject sufficiently large certificates; the mathematical claim concerns ideal finite derivations.

7.3 Measurements and their limitations

All rows count the whole standalone module, including JSON input, validation, scope/substitution routines, proof rules, and output. The normalization protocol is `ast.unparse(ast.parse(source))`

plus one final newline. Significant-token counts exclude comments, indentation, and newline bookkeeping. AST counts include all nodes returned by `ast.walk`. Gzip uses compression level 9 and `mtime=0` on the normalized module, and is informational only.

Complete module	Bytes	Tokens	AST nodes	Gzip 9
Archived seven-rule control	3145	1024	1080	964
Matched seven-rule control	3121	1008	1064	957
Computational equality	3039	959	1026	966

The archived control is the exact `control_seven.py` from the incoming zero-normalization package. The matched control differs only by the proved-formula invariant factoring just described. Thus the matched reduction is 82 normalized bytes, 49 significant tokens, and 38 AST nodes. Its gzip size increases by 9 bytes; that is reported, not optimized away. These measurements establish a controlled implementation saving attributable to the smaller interface.

The repository also contains compact arity-driven encodings with different public-validation and certificate contracts, including a reported 516-byte profile [1]. The 3039-byte checker does not beat that number. A sound and competitive port of Eval into that encoding has not been established. It would require a new certificate design and a scope proof, not just relabeling a rule. We therefore make no global minimum claim for either theory presentations or Python source size.

8 Executable evidence and reproducibility

The archive includes exact bidirectional proof translators. They are untrusted: each input is checked by its source kernel and every output by its target kernel. The end sequent is preserved literally. The old-to-new translator implements the derivations in Sections 3 and 4. The other direction expands Eval into the old computation and transport rules. Neither checker imports a translator.

The recorded verification run passed:

- 38 inherited control fixtures: 14 accepted and 24 rejected;
- all 14 accepted inherited proofs translated to $\mathcal{E}$ and back, with unchanged contexts and conclusions and the $9N$ node bound;
- 24 successor-computation round trips covering zero, successor, and variable predecessors, with nested update bodies;
- five additional arbitrary-template congruence round trips, including repeated holes and holes beneath nested binders;
- 51 independently constructed adversarial cases covering all Eval clauses, scope capture, QE bases, DE eigenvariable escape, DI modes, malformed arities/tags, and Boolean/integer confusion;
- 206 standalone process executions, including normal and optimized Python modes, with no unexpected outcomes.

Finite tests supplement the universal written proofs. They do not prove the metatheorems, certify the Python interpreter, or constitute a new Lean or Rocq verification. The existing Lean statements were inspected at the pinned commit; the upstream project was not rebuilt in this investigation.

From the archive directory, run

```
python3 code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The first command regenerates the measurement table, normalized checkers, gzip files, translated examples, and evidence JSON. A single certificate can be checked with

```
python3 code/checker.py < code/examples/root_reflexivity_zero.json
```

Proof translation is available through

```
python3 code/translate.py old-to-new < old-certificate.json
python3 code/translate.py new-to-old < new-certificate.json
```

9 Relation to the inspected proposals

This report relies on the existing witness DI and the native/iPRA bridge; those are not new contributions. The comparison with the closest alternatives is as follows.

One-context and template-free equality.
The former retains an arbitrary term template. The latter removes that template by global occurrence replacement while retaining computation instructions. Eval uses only the current iterator state and performs one computation at the same inference.

Zero-normal and canonical recursors.
These make computation part of a canonical term grammar with normalizing substitution. Eval retains every raw iterator term and ordinary capture-free substitution. Its exact formula endpoints therefore need no normalization translation.

Initializer-only equality.
That rule changes an iterator's initializer and separately retains successor computation. Eval's successor case instead requires an equality for $I(n, a; u.h)$ itself, immediately exposing $h(t)$. Both computation axioms become derived.

Uniform-predecessor and constructed-computation packets.
Those inspected reports construct successor sources from certificate predecessors while retaining the underlying computation and transport rules. Their predecessor is the counter n. Eval uses an equality of the preceding *value* $I(n, a; u.h)$ and changes the primitive rule basis.

Diagonal guards and target-directed closure.
Those concern introduction modes or the meaning/elimination of closure. The present change leaves both native binders, both meanings, and all four logical rules unchanged.

These are distinctions relative to the inspected repository and incoming packages, not a claim of priority across the proof-theory literature.

10 Further research questions

1. **Formalize the six-rule equivalence in Lean.** Keep the existing raw syntax and define Eval by its three constructors or one partial relation. The equality bootstrap is independent of the

long arithmetic bridge. Reusing `wprf_iff_nprf` should make the new formalization substantially smaller than redoing the iPRA interpretation.

2. **Design a compact certificate format for state evaluation.** The strict forward checker is smaller than its matched control, but the backward compact codec uses a different organization. Can predecessor values be recovered from goals without reintroducing arbitrary templates or enlarging scope machinery?
3. **Obtain six rules with direct substitution for the entire calculus.** Eval is now stable, while witness-sensitive DI retains its old mode-change issue. Combine state evaluation with an appropriate substitution-stable introduction scheme and measure the full result.
4. **Reduce the eight-node congruence overhead.** The displayed macro preserves each nontrivial premise once. Determine lower bounds for this equality fragment or a shorter general macro, distinguishing tree proofs from DAG proofs.
5. **Find a precise simplicity order.** Rule counts, syntax constructors, complete checker tokens, source bytes, and certificate sizes measure different costs. A universal minimum is not established here; a declared admissible representation class would make stronger optimality questions meaningful.

A The complete selected checker

The following is the complete program, not an illustrative fragment. Its measurements include all of it. The code is intentionally short but retains public syntax and scope validation.

```
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def term(a, n):
    match a:
        case ['v', i]:
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')


def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
```

```
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')


def walk(a, f, d=0):
    if a[0] == 'v':
        return f(a[1], d)
    return [a[0]] + [walk(b, f, d + ({'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a)
    ))[j])
                     for j, b in enumerate(a[1:])]


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])


def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop
    ])


def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['refl', a]:
            term(a, n)
            return ['=', a, a]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = a if a[0] == '=' and t == ['z'] else shift(sub(a, ['z']))
            require(infer(p, [h] + [shift(x) for x in g], n + 1) == sub(a, ['s', ['v', 0]], 0))
            return ['d', b]
        case ['ev', p, q]:
            _, s, t = infer(p, g, n)
            _, b, r = infer(q, g, n)
            if b != s:
                _, c, a, h = b
                if c != ['z']:
                    require(c[0] == 's')
                    a, t = ['i', c[1], a, h], sub(h, t)
```

```
                require(a == s)
            return ['=', r, t]
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

References

[1] V. Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README, frozen repository commit stated in Section 1. https://github.com/VladimirReshetnikov/ProveIt/tree/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, *NativeOneIntro.lean*, especially `WPrf` and `wprf_iff_nprf`, same commit. https://github.com/VladimirReshetnikov/ProveIt/blob/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeOneIntro.lean.

[3] ProveIt, *NativeBiInterpretation.lean* and *BiInterpretation.lean*, same commit: faithful translations, both formula round trips, and equiconsistency. https://github.com/VladimirReshetnikov/ProveIt/blob/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeBiInterpretation.lean.

[4] ProveIt, *ConciseIPRA/docs/incoming*, same commit. The six inspected packages are diagonal guard, initializer equality, scope kernel, target closure, template-free equality, and zero-normal syntax. Exact archive names and file provenance are recorded in the accompanying `evidence/provenance.json`. https://github.com/VladimirReshetnikov/ProveIt/tree/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming.